

Hadoop Distributed File System

Requirements/Features for a DFS

- It is a **distributed file system**
 - Manages storage across a network of machines in a cluster
- Designed to run on clusters of **commodity hardware**
 - Does not require expensive, highly-reliable hardware
 - Commonly available, low-cost hardware
- Highly **fault-tolerant**
 - Failures are the norm rather than the exception

Organization of a DFS

- Files are divided into **chunks** (or blocks)
 - typically 64/128 megabytes in size
- Blocks are replicated at different compute nodes (usually 3+)
- Nodes holding copies of one block are located on different racks
- Block size and the degree of replication can be decided by the user
- A special node (the master node) stores, for each file, the positions of its blocks
- The master node is itself replicated

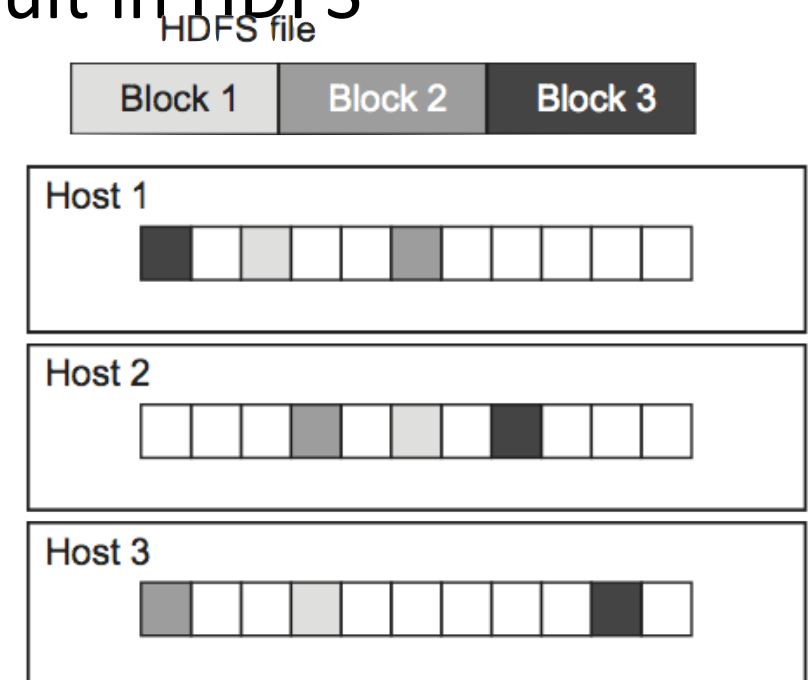
Blocks

Single-disk filesystems

- Minimum amount of data that it can read or write
- File System Blocks are typically few KB
- Disk blocks are normally 512 bytes

DFS

- DFS Block is much larger – e.g., 64/128 MB by default in HDFS
 - Large to minimize the cost of seeks
- Block abstractions allows
 - A file can be larger than any single disk
 - Using blocks as units of abstraction simplifies the storage subsystem (e.g., fixed size)



Namenodes & Datanodes

- **Master/slave** architecture
- HDFS cluster consists of a **single Namenode**, a master server that manages the file system namespace and regulates access to files by clients.
 - Maintains filesystem tree
 - Files metadata
 - File-to-block mapping
 - Location of blocks (i.e., on which datanodes)
 - Access permissions
- There are **a number of Datanodes**, usually one per node in a cluster. Datanodes store and manage the actual data blocks.
 - A file is split into one or more blocks, and blocks are stored in Datanodes.
 - The Datanodes manage storage attached to the nodes that they run on.
 - Datanodes serve read/write requests, perform block creation, deletion, and replication upon instruction from Namenode.
 - Each Datanode sends a heartbeat message periodically to the Namenode.

writing operations

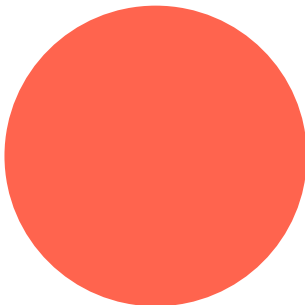
HDFS Architecture

Client



a client component, which is an interface (not actually a client where you write, not a map task, a middle ware iin between the task and the component. It's something which stays in between the client and the HDFS compoonent. Periodically the node send an information to the main node, telling that they are healty and alive, ready to respond to any kind of write ead operation.

Namenode



Datanode A



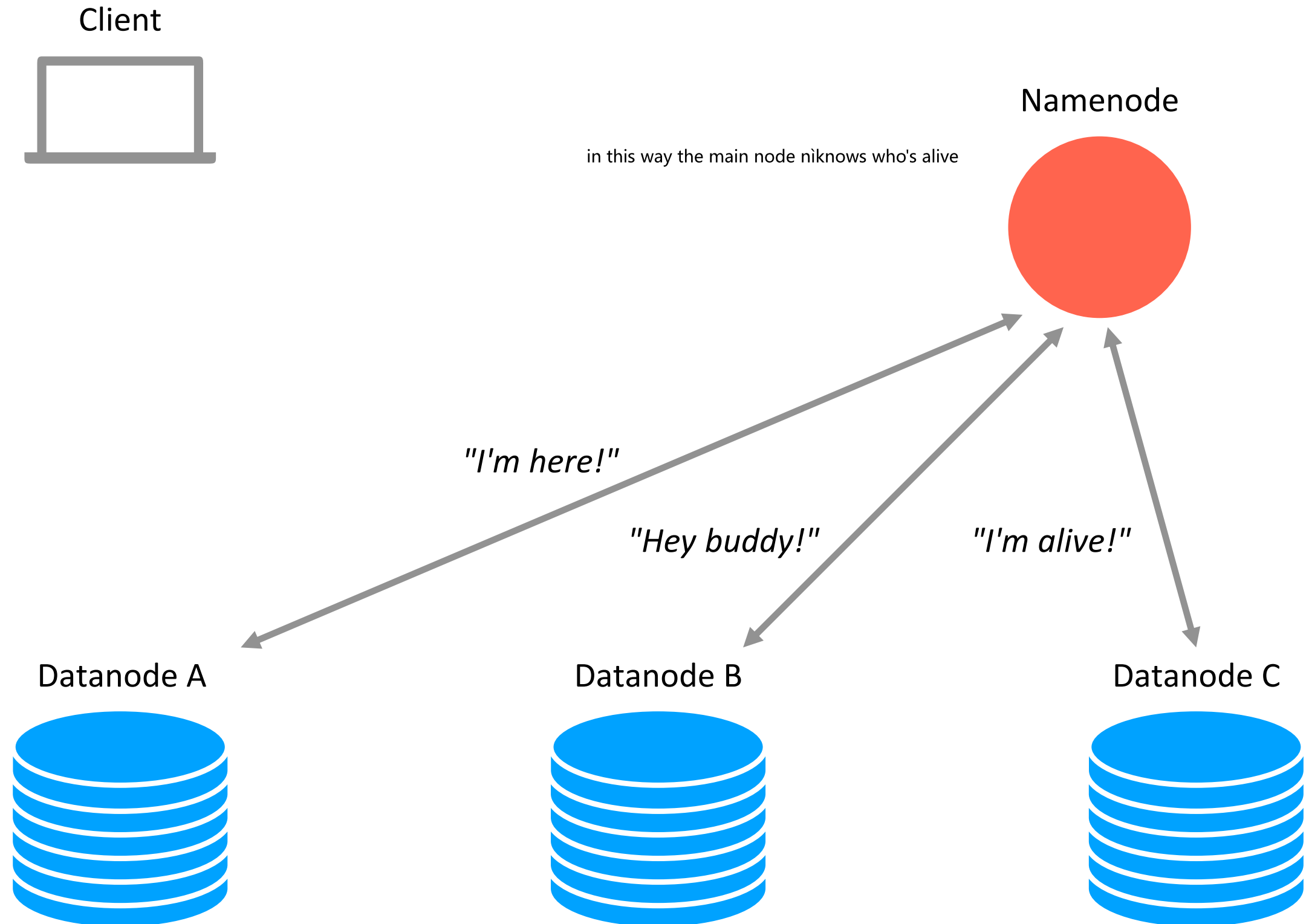
Datanode B



Datanode C



HDFS Architecture

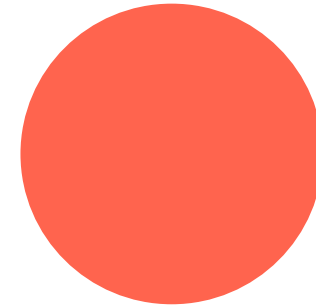


HDFS Architecture

Client



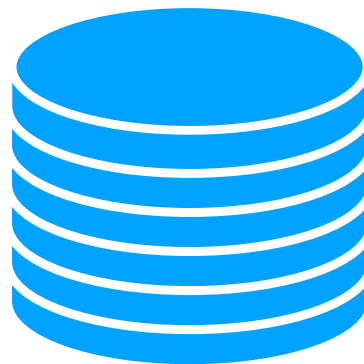
Namenode



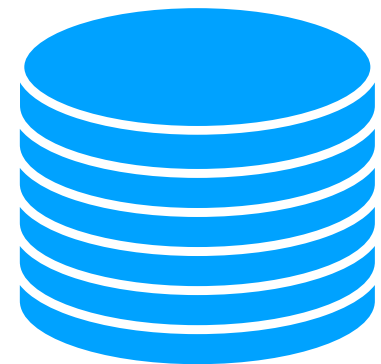
Datanode A



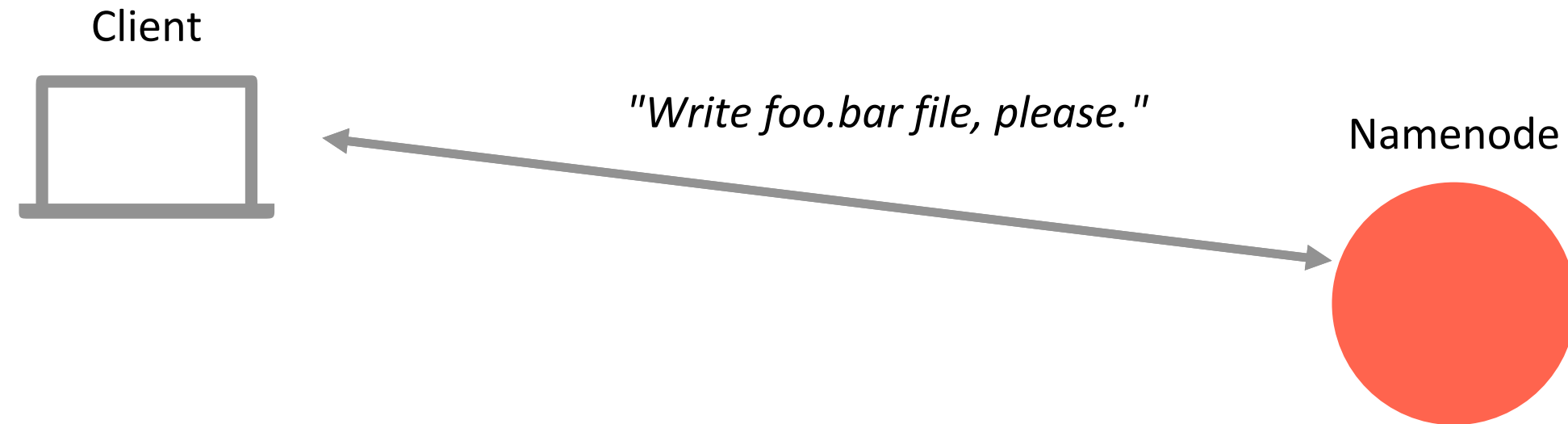
Datanode B



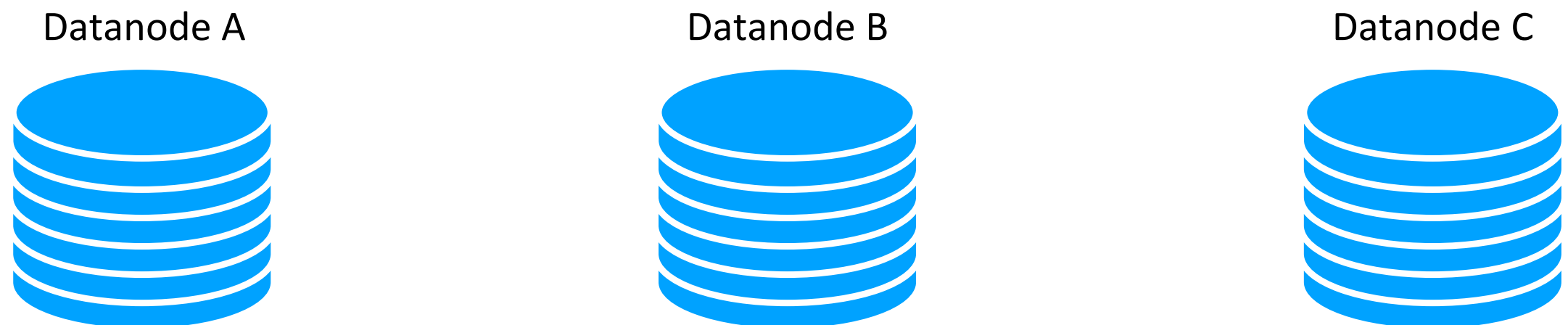
Datanode C



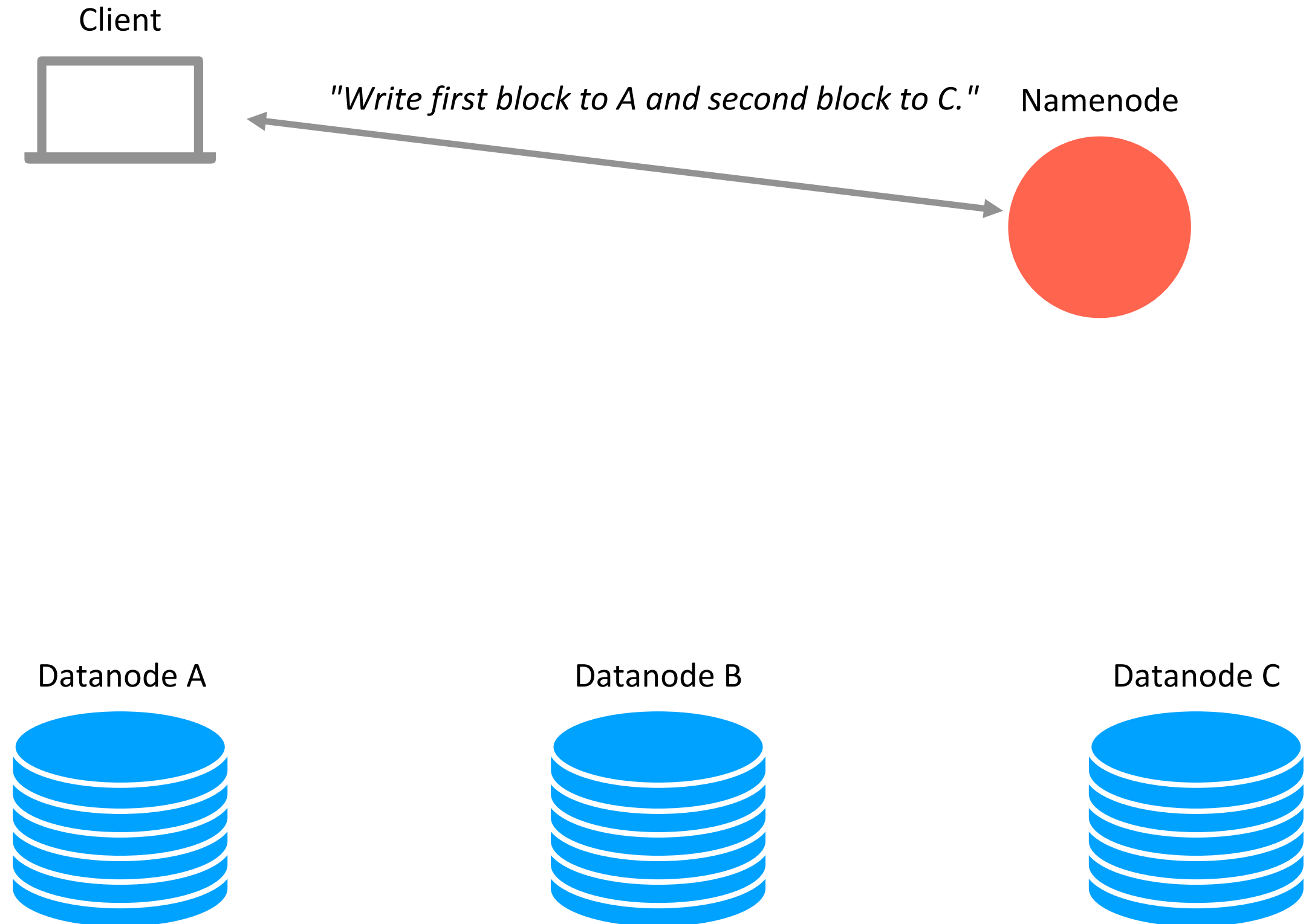
HDFS Architecture



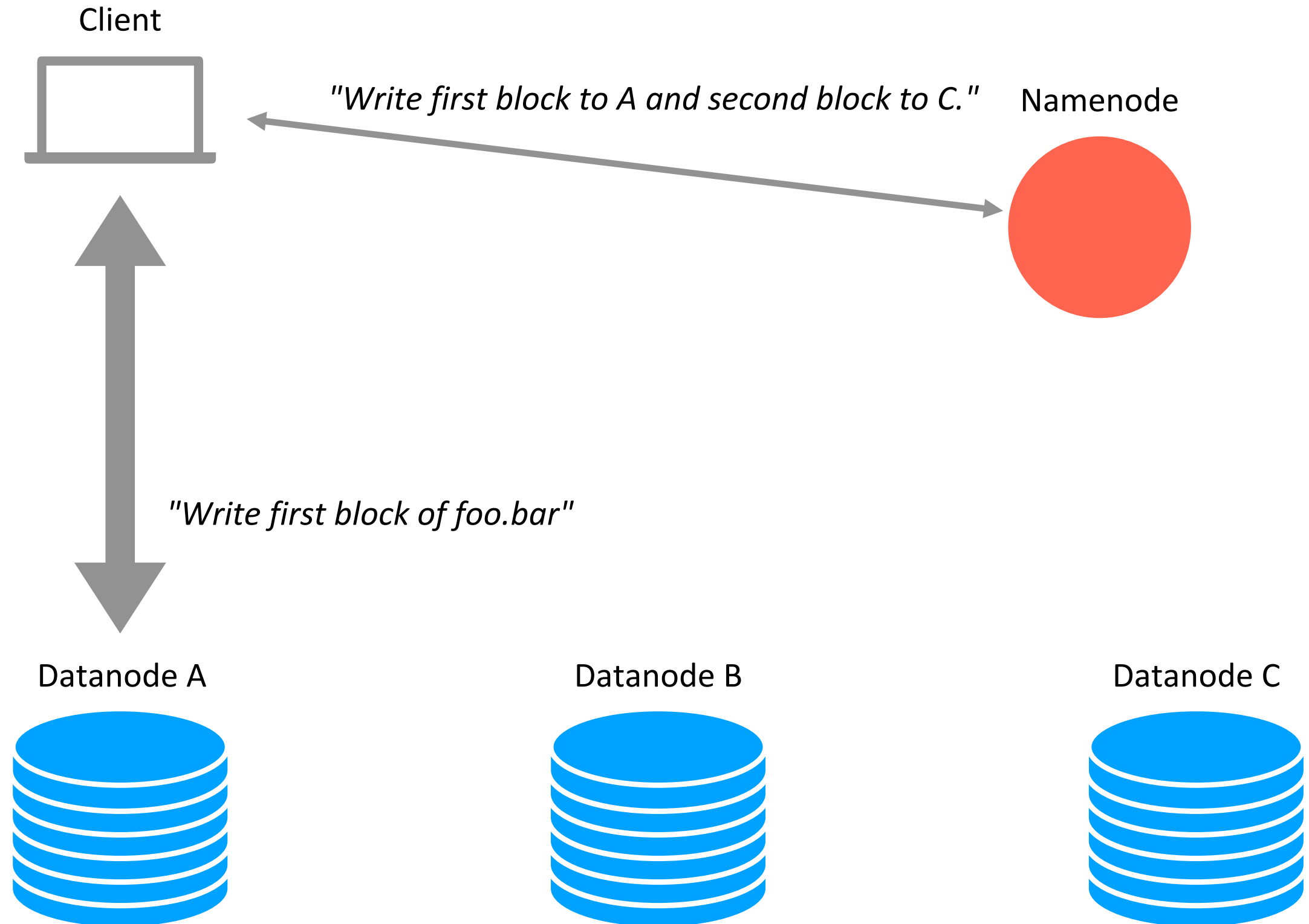
we assume that this file is bigger than the size of a single block, so the node after this request is performed a request to understand if the client has the permission to do whatever. Afterwards the node will tell what to do. The client will request to write a certain block



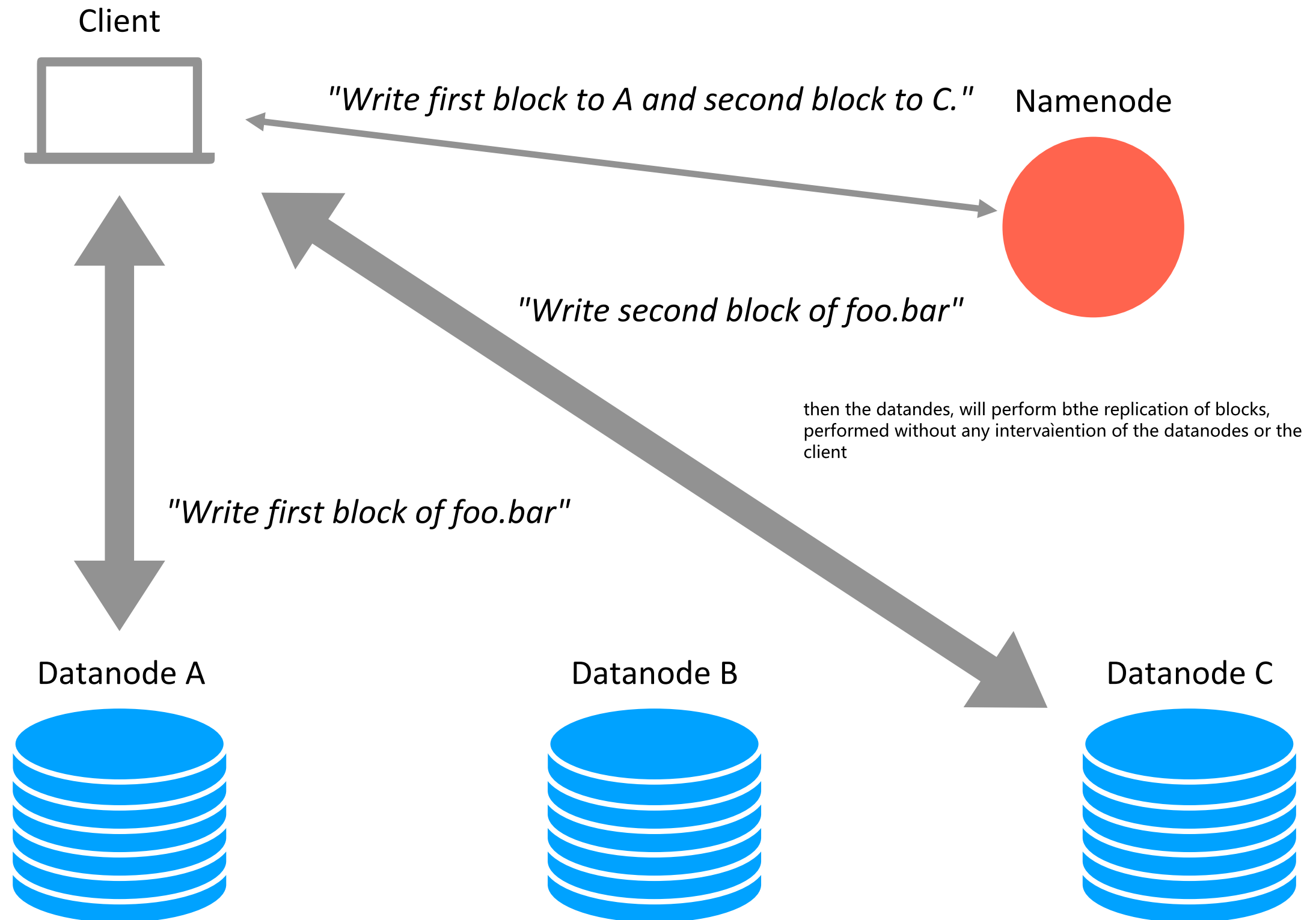
HDFS Architecture



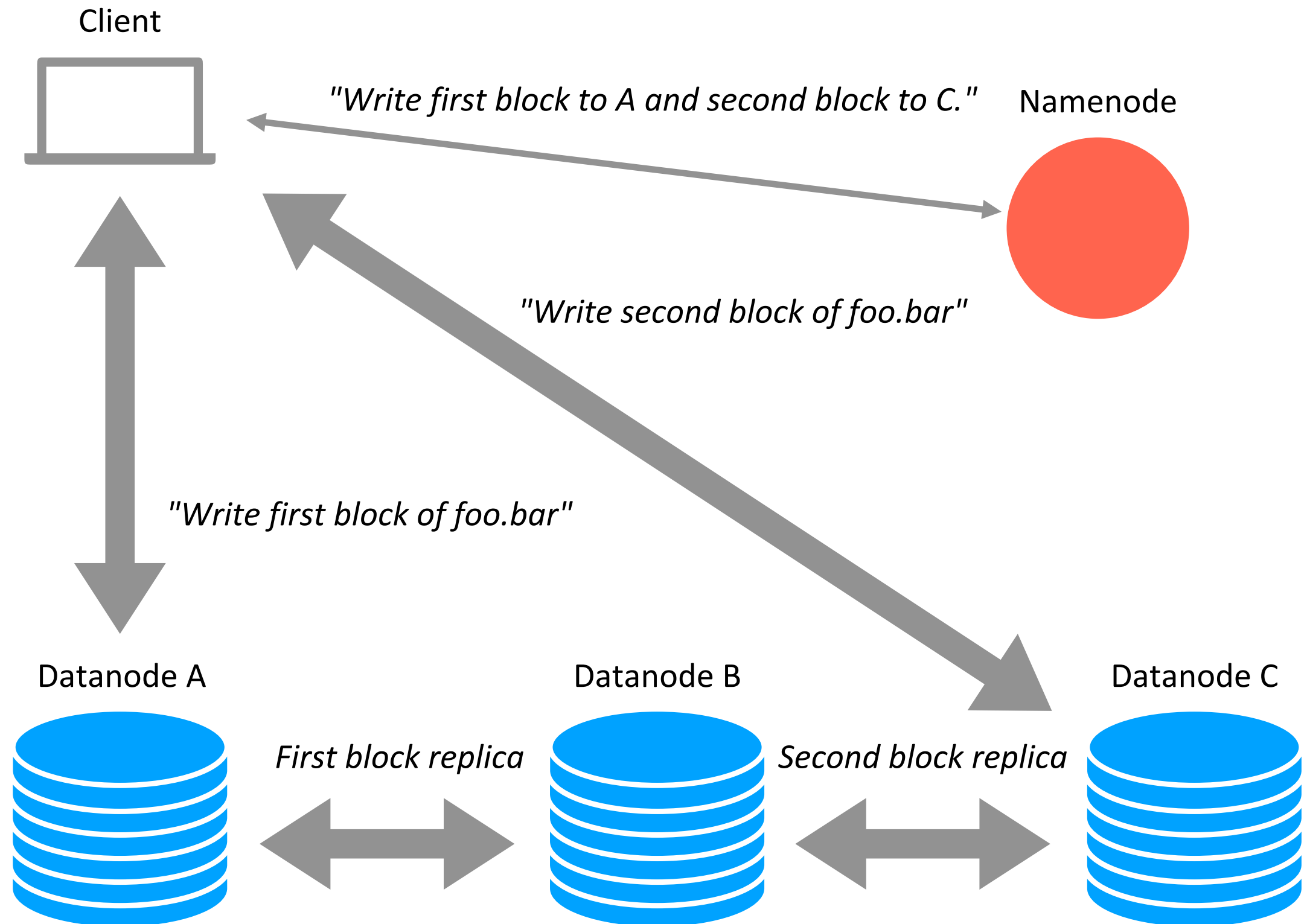
HDFS Architecture



HDFS Architecture

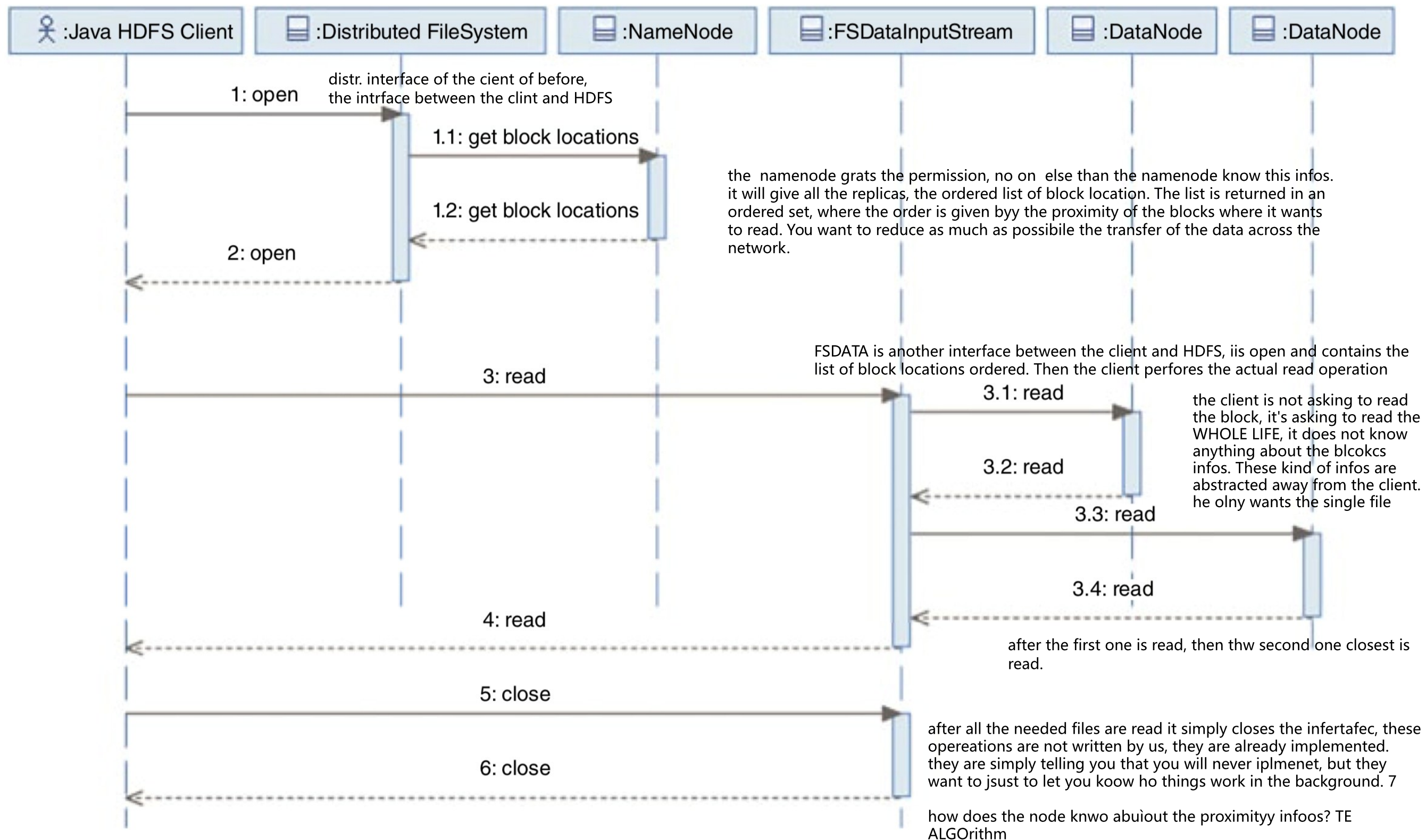


HDFS Architecture



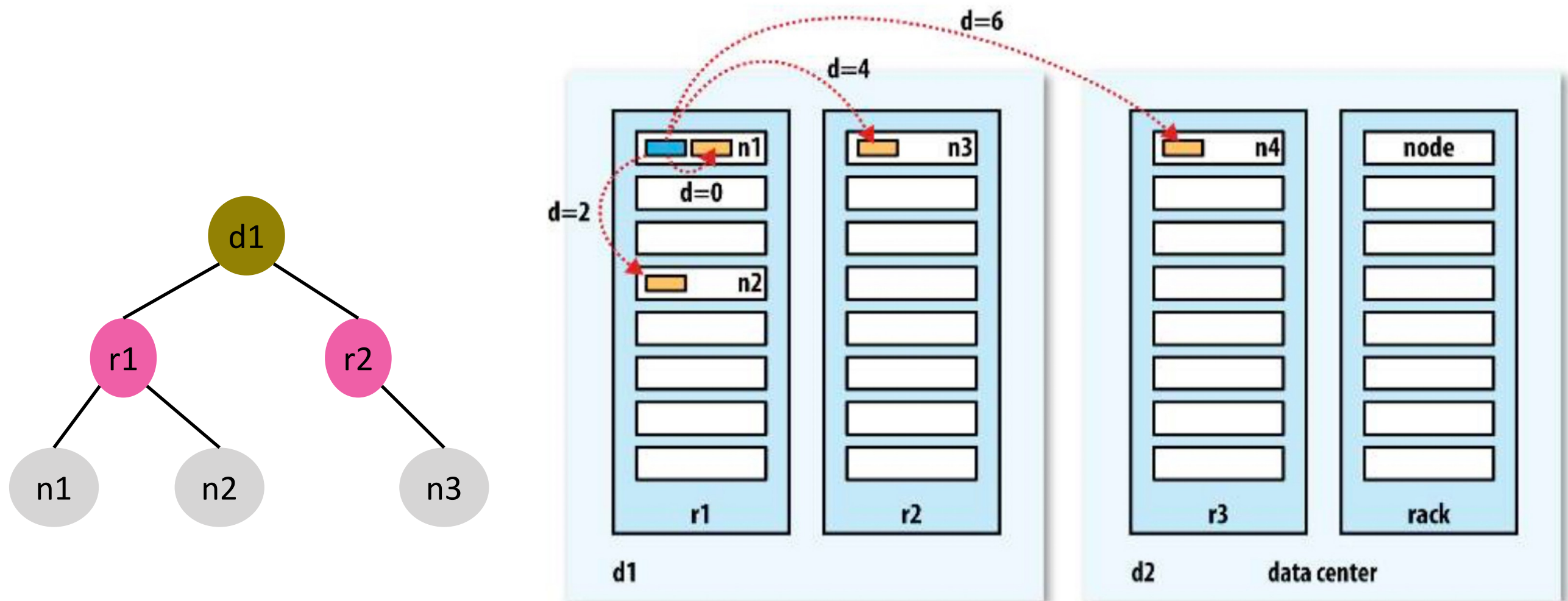
Anatomy of a read

client= actual client where you write. --> maptask, performed on an input, which is the input split installed on HDFS coso.

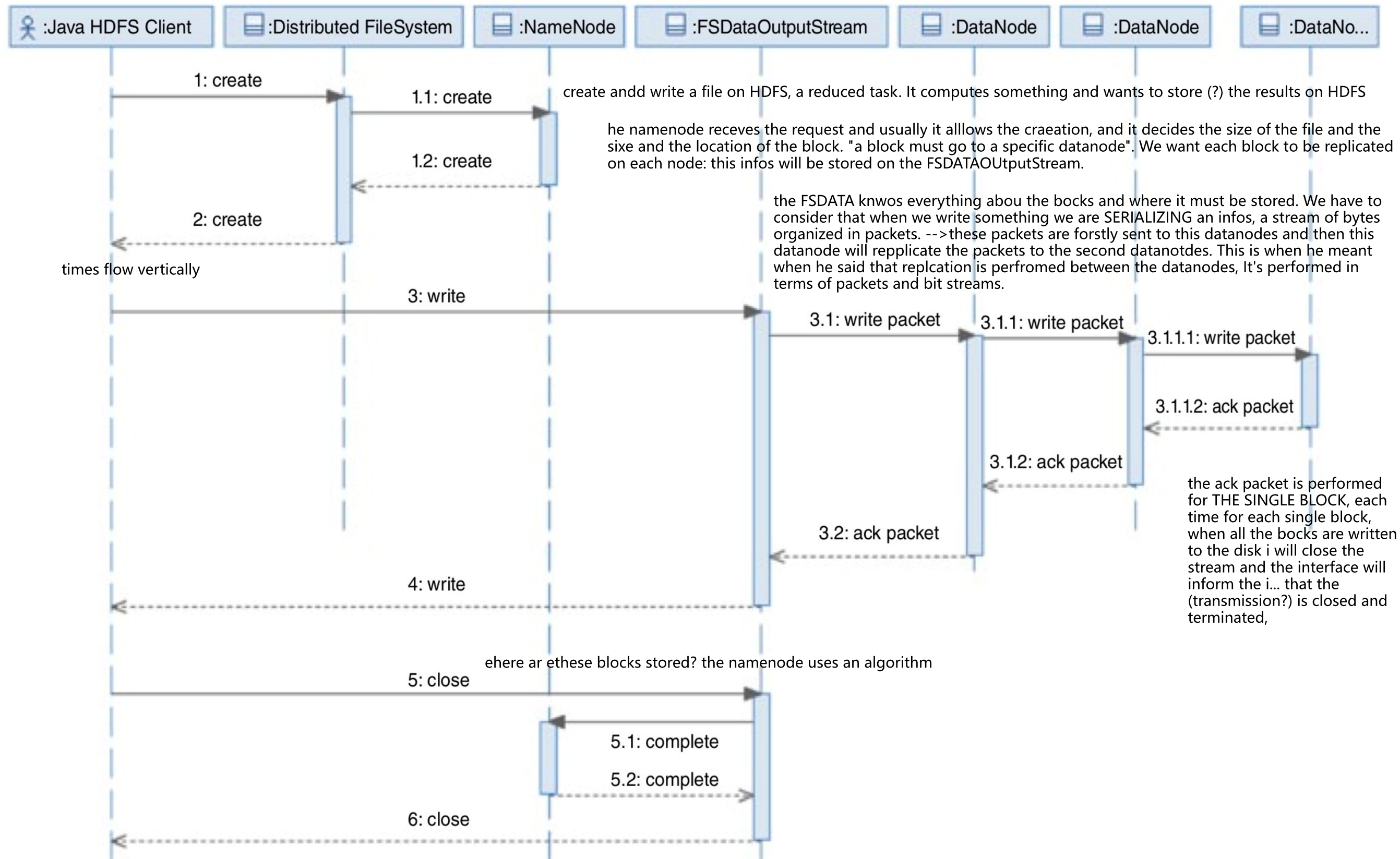


Network Topology and Hadoop

Hadoop approach – The network is represented as a tree and the distance between two nodes is the sum of their distances to their closest common ancestor.



Anatomy of a write



Replication on Datanodes

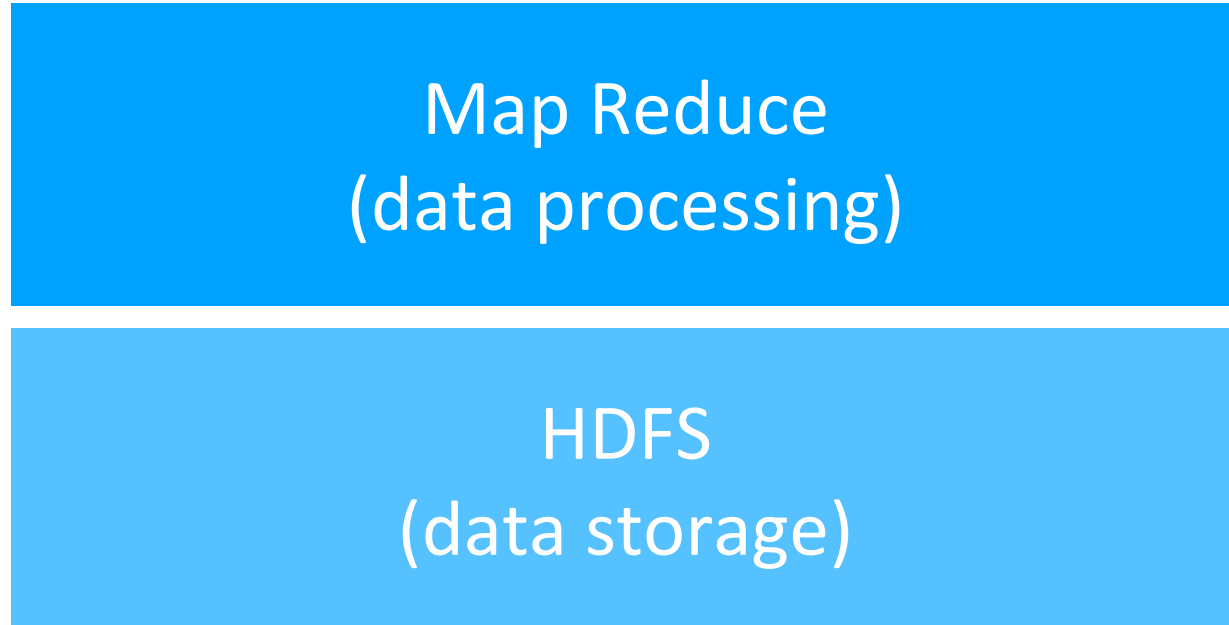
How does the namenode choose which datanodes to store the replicas on?

- First replica on the same node as the client
 - For clients running outside the cluster, a node is chosen at random
- Second replica on a different rack from the first, chosen at random
- Third replica on the same rack as the second, but on a different node chosen at random
- Further replicas are placed on random nodes in the cluster
- The system always tries to avoid placing too many replicas on the same rack/node

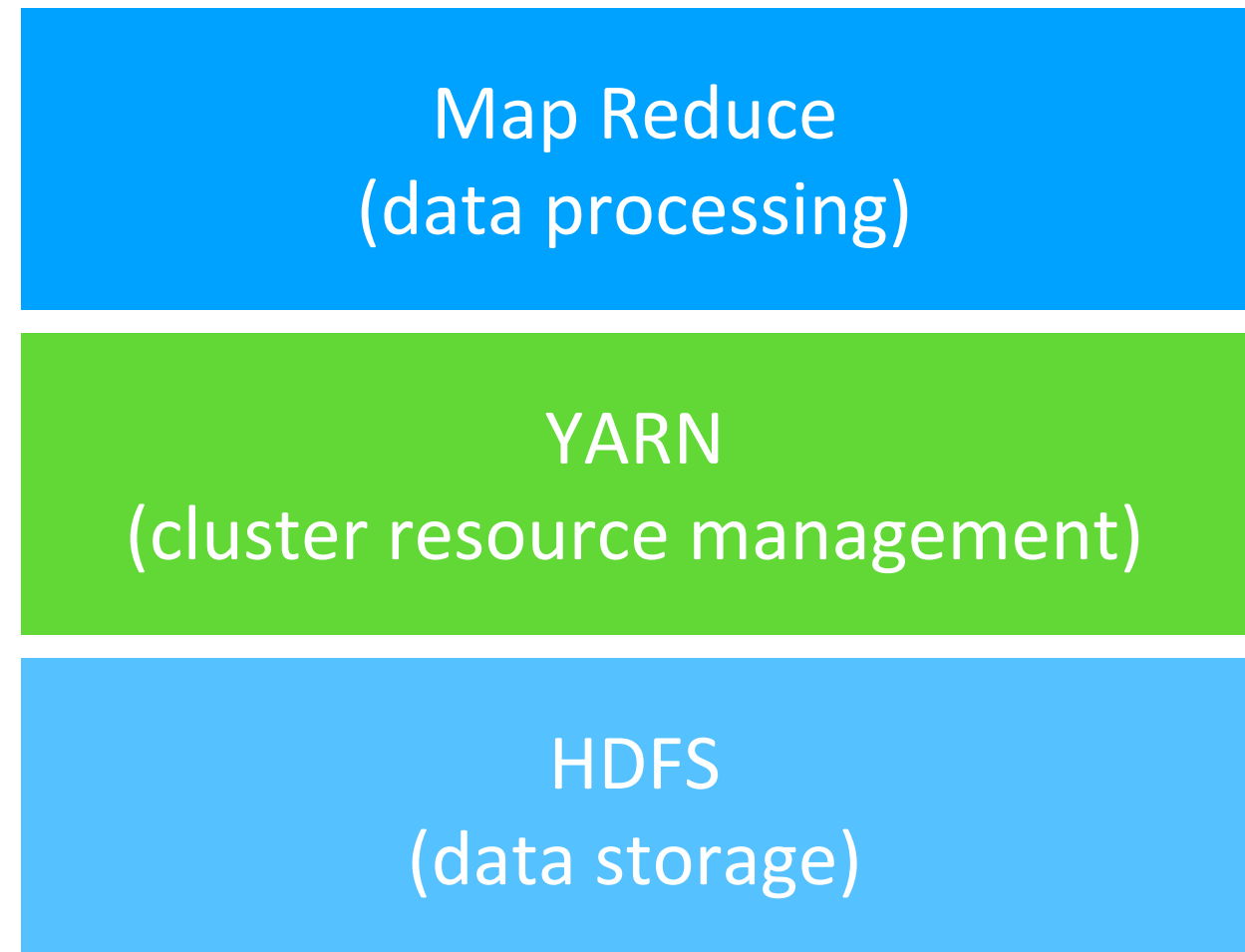
Hadoop Distributed Resource Management

Resource management in Hadoop versions

Hadoop 1.0



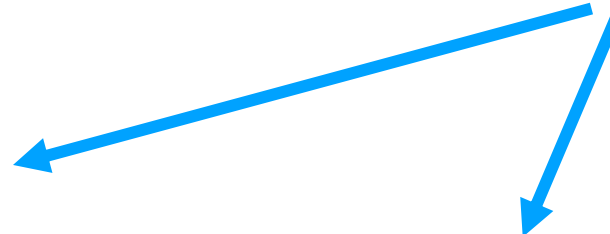
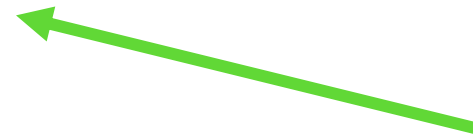
Hadoop 2.0, 3.0



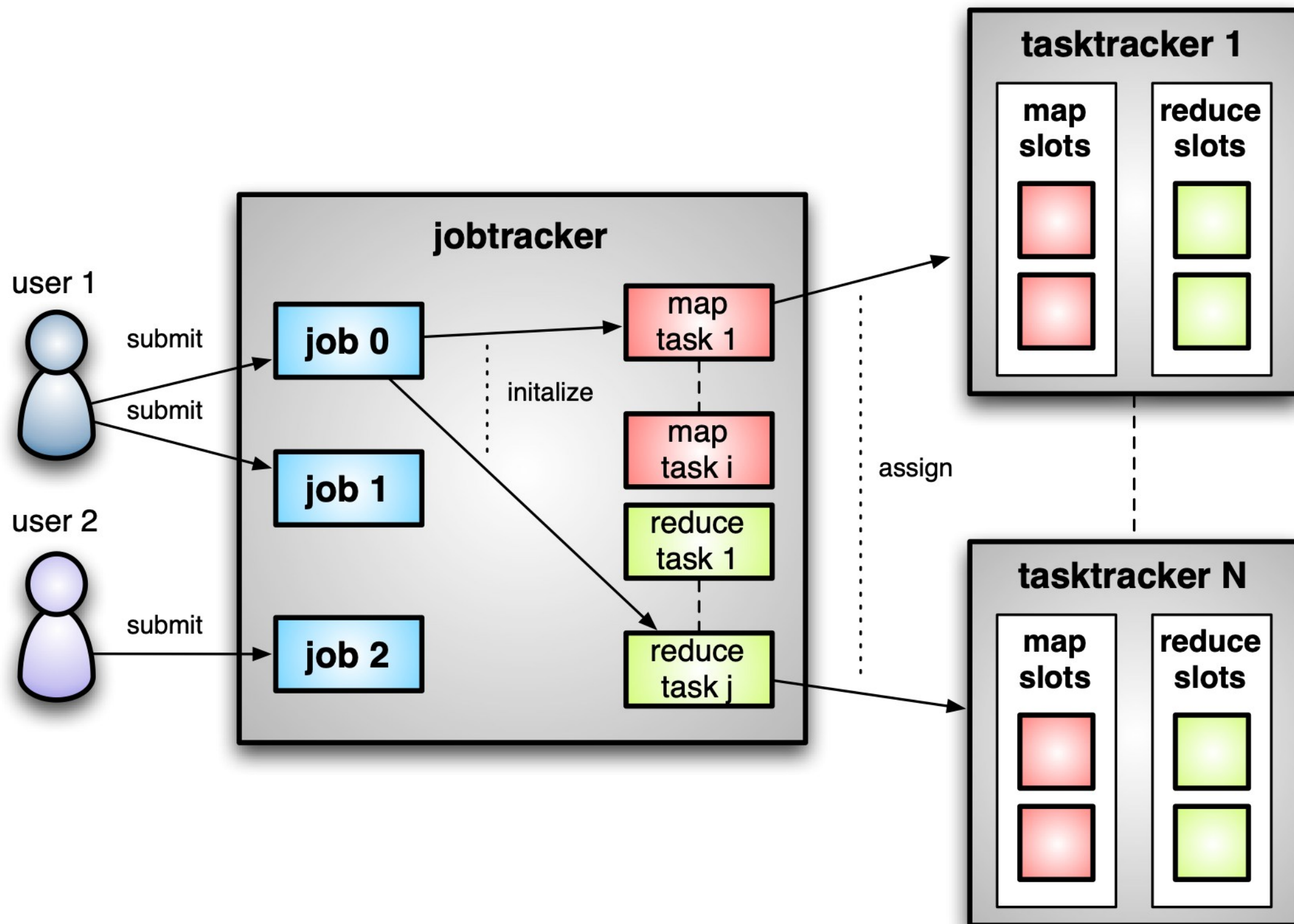
Hadoop 1.0

- Job
 - Unit of work that the client wants to be performed
- Task
 - Unit of work that Hadoop schedules and runs on nodes in the cluster (map & reduce)
- Slot
 - Processing element for tasks (map & reduce slots)
- Job Tracker
 - Accepts jobs submitted by users
 - Creates tasks
 - Assigns map and reduce tasks to Task Trackers
 - Monitors tasks and Task Trackers status, keeping a record of progress for each job and re-executing tasks upon failure
- Task Tracker
 - Runs map and reduce tasks upon instruction from the Job Tracker
 - Manages storage and transmission of intermediate output
 - Sends progress reports to the Job Tracker

These are the main problems in Hadoop 1.0



Hadoop 1.0



Hadoop 1.0 Limitations

- Scalability
 - Job tracker is a bottleneck
 - Job Tracker performs resource allocation and monitoring for all the jobs
 - No more than 4,000 nodes and 40,000 concurrent tasks (whereas with YARN, it goes up to 10,000 nodes and 100,000 tasks)
- Availability
 - Job tracker is a single point of failure
 - Any failure kills all queued and running jobs
 - Replicating the state of this component to achieve availability can be complex
- Resource Utilization
 - Due to the predefined tag of map and reduce slots for each Task Tracker, utilization issues occur, e.g., a reduce task has to wait because only map slots are available in the cluster
 - Furthermore, a slot can be too big (waste of resources) or too small (which may cause a failure) for a particular task

YARN Components

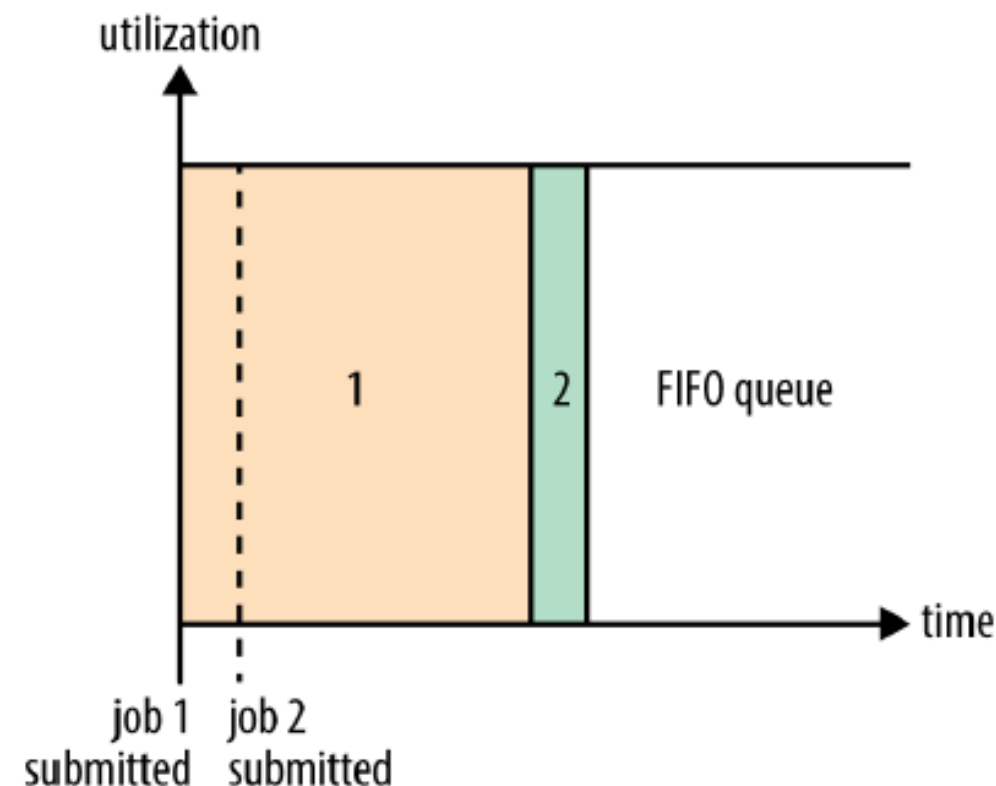
- YARN provides its core services via two types of long-running daemons
 - a Resource Manager (one per cluster) to manage the use of resources across the cluster
 - Node Managers (one per node in the cluster) to launch containers and monitor usage of container resources, reporting stats to resource manager
- A container is a set of computer resources allocated to run an application-specific process (e.g., a map or reduce task).
- A resource request for a set of containers can express
 - the amount of computer resources required for each container (memory and CPU)
 - locality constraints for the containers in that request (e.g., allocate container on a node where there is a replica of the HDFS block)
- If the locality constraint cannot be met
 - no allocation is made or
 - the constraint can be loosened (e.g., on another node in the same rack)

YARN Scheduling (I)

- In the real world, cluster resources are limited
 - Scheduling (i.e., allocating resources to applications) is a complex problem and there is no one “best” policy
 - YARN provides different schedulers that allocate resources to applications based on different policies

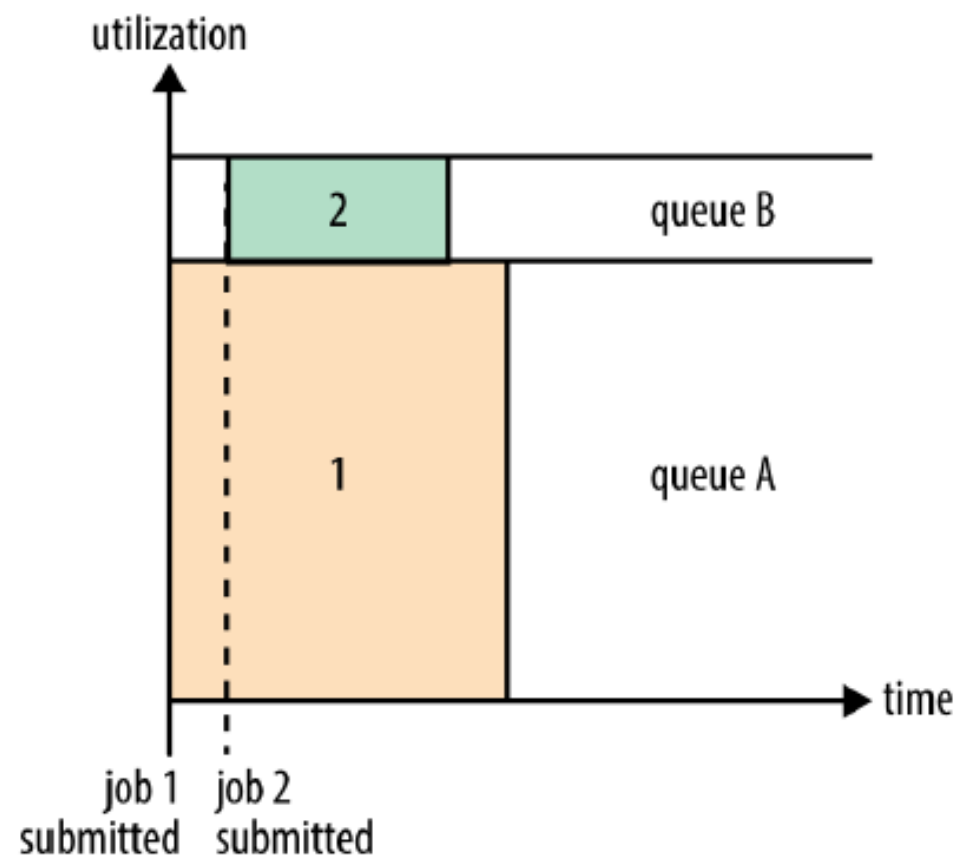
YARN Scheduling (II): FIFO Scheduler

- The FIFO Scheduler places applications in a single queue and runs them in the order of submission (first in, first out)
 - Requests for the first application in the queue are allocated first; once its requests have been satisfied, the next application in the queue is served
 - Simple to understand and no configuration needed
 - Not suitable for shared clusters. Large applications will use all the resources in a cluster, so other applications have to wait their turn



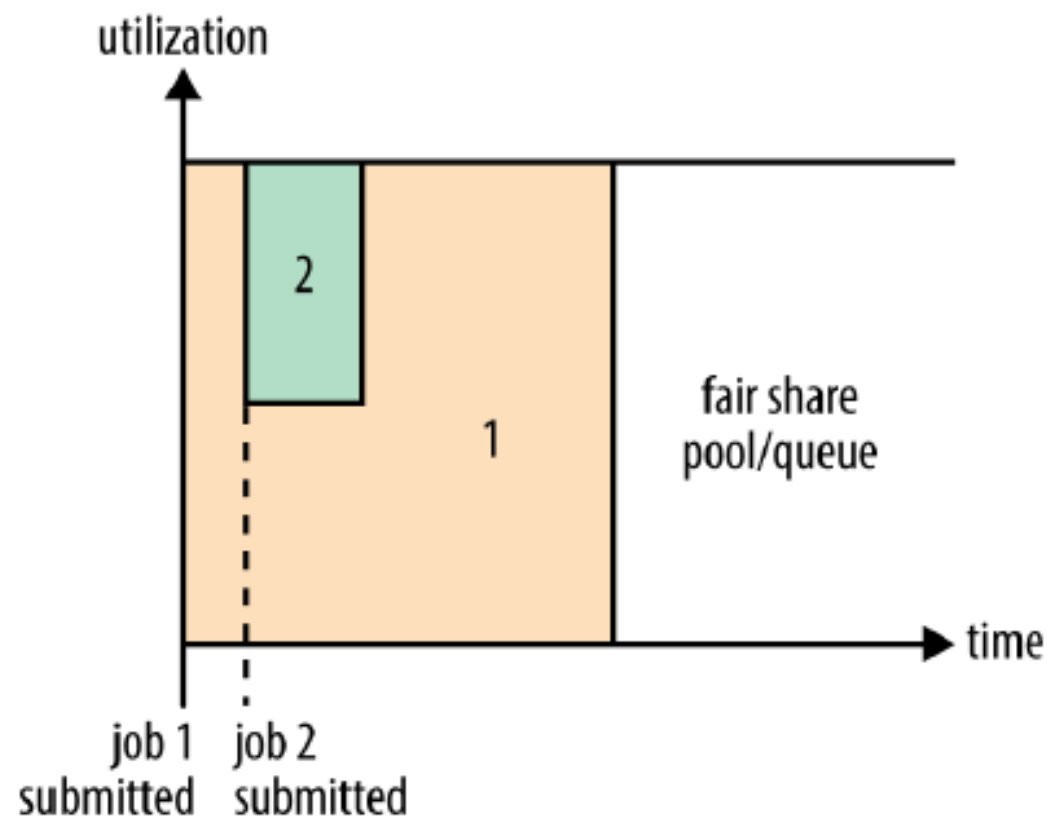
YARN Scheduling (III): Capacity Scheduler

- The Capacity Scheduler has separate queues
 - Each queue dedicated to an organization accessing the cluster
 - Each queue configured to use a given fraction of the cluster capacity
 - Within a queue, applications are scheduled using FIFO scheduling
 - A large job finishes later than when using FIFO Scheduler



YARN Scheduling (IV): Fair Scheduler

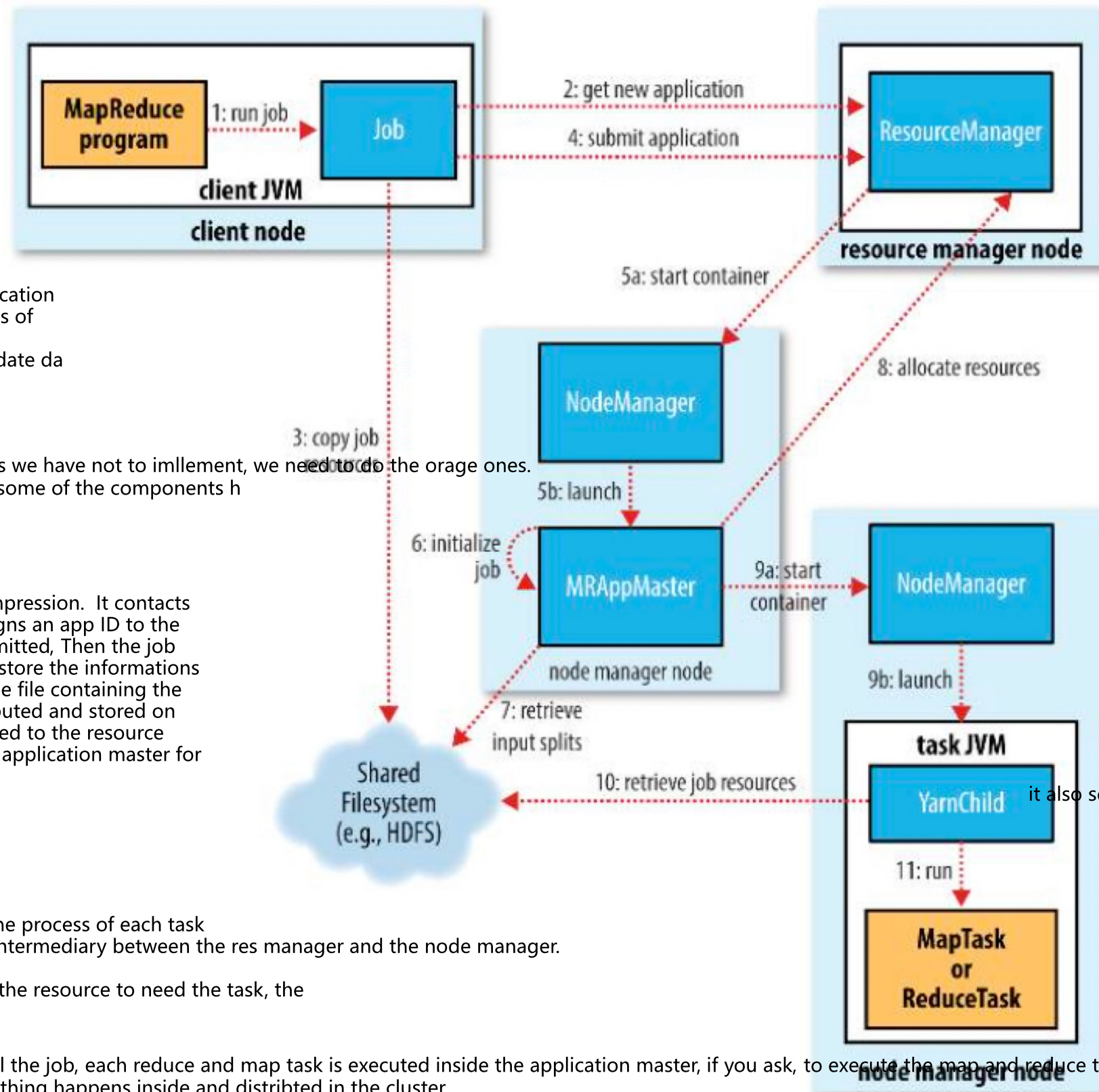
- The Fair Scheduler
 - There is no need to reserve a set amount of capacity
 - It will dynamically balance resources among all running jobs
 - E.g., when the first job starts, it gets all cluster resources (it is the only one running). When the second job starts, it gets half of the cluster resources, and so on
 - Fair scheduling is per-user: if user A submits two jobs and user B only one, then each A's job gets 25% while B's job gets 50% of resources



YARN Scheduling (V): Delay Scheduling

- All YARN schedulers try to honor locality requests.
- On a busy cluster, however, there is a good chance that a required node is saturated by other containers at the time of a request
- As seen, we can loosen the constraint by allocating resources on another node on the same rack
- In practice, waiting a short time (no more than a few seconds) can dramatically increase the chances of being allocated a container on the requested node, and therefore increase the efficiency of the cluster
- This feature is called Delay scheduling and is supported by both the Capacity Scheduler and the Fair Scheduler
- Every node manager in a YARN cluster periodically sends a heartbeat request to the resource manager
 - Heartbeats carry information about the node manager's running containers and the resources available for new containers
 - Each heartbeat is a potential scheduling opportunity for an application to run a container
 - When using delay scheduling, the scheduler doesn't simply use the first scheduling opportunity it receives, but waits for up to a given maximum number of scheduling opportunities to occur before loosening the locality constraint

Anatomy of a MapReduce application run



what happens in how the application works. First of all, we have loads of components, All the blue components are offered and data da hadoop manager

all the blue things are things we have not to implement, we need to do the orange ones. the concept of abstraction, some of the components h

job: java class with waitforcompression. It contacts the resource manager, which assigns an app ID to the job which is going to be submitted, Then the job component goes to HDFS to store the information needed to execute the job, the file containing the input strings, which are computed and stored on HDFS. The app is not submitted to the resource manager, which launches the application master for each job.

the app master do:

- periodically monitoring the process of each task
- behaves as a broker, an intermediary between the resource manager and the node manager.

The app master contacts the resource to need the task, the

it can ask to execute a map task, the manager will start a java VM, which will start a YarnChild

it also sends heartbeats to the MRAPPMASTER

the USER is, in this case, that all the job, each reduce and map task is executed inside the application master, if you ask, to execute the map and reduce tasks to execute 'em inside himself, but does not happen usually, everything happens inside and distributed in the cluster

Fault Tolerance (I)

- Task Failure
 - The user code in the map or reduce task throws a runtime exception
 - the task JVM reports the error back to its parent application master before it exits.
 - The application master marks the task attempt as failed
 - The application master frees up the container
 - Sudden exit of the task JVM
 - the node manager informs the application master
 - The application master marks the task attempt as failed
 - The application master frees up the container
 - Hanging tasks
 - The application master notices that it hasn't received a progress update for a while
 - The task JVM process will be killed automatically after this period (typically 10 minutes)
 - The application master marks the task as failed
 - When the application master is notified of a task attempt that has failed, it will reschedule execution of the task
 - The application master will try to avoid rescheduling the task on a node manager where it has previously failed

Fault Tolerance (II)

- Application Master Failure
 - Application master periodically sends heartbeats to the resource manager.
 - The resource manager will detect the failure and start a new instance of the master running in a new container (managed by a node manager)
 - The client needs to go back to the resource manager to ask for the new application master's address (this process is transparent to the user)
 - If a MapReduce application master fails twice it will not be tried again and the job will fail
- Node Manager Failure
 - Node managers periodically send heartbeats to the resource manager
 - The resource manager will notice a node manager that has stopped sending heartbeats if it hasn't received one for 10 minutes
 - The resource manager will remove it from its pool of nodes to schedule containers on
 - Any task or application master running on the failed node manager will be considered failed and will be therefore recovered by using the mechanisms seen before
 - Node managers may be blacklisted if the number of failures for the application is high (even if node manager itself has not failed)
 - Blacklisting is done by the application master

Fault Tolerance (III)

- Resource Manager Failure
 - Failure of the resource manager is serious: without it neither jobs nor task containers can be launched
 - In the default configuration, the resource manager is a single point of failure
 - To achieve high availability, it is necessary to run a pair of resource managers in an active-standby configuration
 - Information about all the running applications is stored in a highly available state store
 - The standby resource manager can recover the core state of the failed active resource manager
 - The transition of a resource manager from standby to active is handled by a failover controller
 - The default failover controller uses leader election mechanism to ensure that there is only a single active resource manager at one time
 - Clients and node managers must be configured to handle resource manager failover. They try connecting to each resource manager in a round-robin fashion until they find the active one.

Speculative Execution

- Problem: Stragglers (i.e., slow tasks) significantly lengthen the completion time of the job
- Tasks can be slow due to, e.g.,
 - Hardware degradation
 - Software misconfiguration
- Solution: Speculative execution
 - Application master tracks the progress of all tasks of the same type (Map and Reduce) in a job
 - Launches speculative duplicate tasks for the small portion of tasks that are running significantly slower than the average
 - Whichever one (i.e., original and speculative) finishes first, “wins”. The other is killed.
- Additional cost: a few percent more resource usage

Shuffle and Sort

